Lab 27 — Storage Resilience with Software RAID (mdadm)

> **SY0-701 Objective 3.4** — Explain the importance of resilience and recovery in security architecture
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~30 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: Build a RAID 1 mirror from virtual disks, fail a disk and prove the data survives, rebuild onto a replacement, compare RAID 5 capacity, and show why RAID is not a backup.

You'll build: A RAID 1 array (`/dev/md10`) that survives a failure and rebuilds, a three-disk RAID 5 array (`/dev/md11`), recorded checksums, and a saved array configuration.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 26 (backups) is useful context; tools installed in step 1. Six 128 MB files on loop devices stand in for physical disks.

Background

- **High availability** removes single points of failure. At the storage layer that means **RAID**: RAID 1 mirrors, RAID 5 uses distributed parity (survives one failure), RAID 6 uses double parity (survives two), RAID 10 is a stripe of mirrors. RAID 0 stripes for speed with **no** redundancy.
- A **degraded** array keeps serving data after a member fails. Replacing the disk triggers a **rebuild**, during which a second failure is the biggest risk — so arrays are monitored and spares are kept ready.
- RAID trades capacity for resilience: a two-disk mirror gives half the raw space; a three-disk RAID 5 gives two-thirds.
- **RAID is not a backup.** It protects against hardware failure only; deletion, corruption and ransomware are written to every member instantly. Backups (Lab 26), replication and journaling cover the rest.
- Resilience is wider than disks: dual power supplies, UPS and generators, clustering, load balancing and geographically dispersed sites apply the same idea to every layer.

Steps

1. Install mdadm and create virtual disks

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y mdadm
modprobe raid1 2>/dev/null; modprobe raid456 2>/dev/null; true
mkdir -p /root/lab27 && cd /root/lab27
for d in 1 2 3 4 5 6; do truncate -s 128M disk$d.img; done
ls -lh /root/lab27/*.img
```

2. Attach the images as loop devices

The device names are saved to a file so later steps (and new terminals) can reuse them.

```
cd /root/lab27
for d in 1 2 3 4 5 6; do losetup -f --show disk$d.img; done | tee loops.txt
mapfile -t LO < loops.txt
echo "Mirror: ${LO[0]} ${LO[1]}  Spare: ${LO[2]}  RAID5: ${LO[3]} ${LO[4]} ${LO[5]}"
```

3. Build a RAID 1 mirror and watch it sync

`--run` answers the metadata confirmation prompt so the command is non-interactive.

```
mapfile -t LO < /root/lab27/loops.txt
yes | mdadm --create /dev/md10 --level=1 --raid-devices=2 --run "${LO[0]}" "${LO[1]}"
cat /proc/mdstat
mdadm --wait /dev/md10 2>/dev/null; true
mdadm --detail /dev/md10 | grep -E 'Raid Level|Array Size|State :|Active Devices'
```

4. Create a filesystem and record checksums

Store a large file and a small ledger, and save their SHA-256 hashes as the integrity baseline.

```
mkfs.ext4 -q /dev/md10
mkdir -p /mnt/raid && mount /dev/md10 /mnt/raid
head -c 20M /dev/urandom > /mnt/raid/database.bin
echo "IMPORTANT synthetic ledger - balance 1,000,000" > /mnt/raid/ledger.txt
sha256sum /mnt/raid/database.bin /mnt/raid/ledger.txt | tee /root/lab27/before.sha256
```

5. Fail a disk and prove the data survives

Mark one member as faulty, remove it, flush the page cache so reads really come from the remaining disk, and verify the checksums.

```
mapfile -t LO < /root/lab27/loops.txt
mdadm /dev/md10 --fail "${LO[0]}"
mdadm /dev/md10 --remove "${LO[0]}"
mdadm --detail /dev/md10 | grep -E 'State :|Active Devices|Failed Devices'
cat /proc/mdstat
sync && echo 3 > /proc/sys/vm/drop_caches
sha256sum -c /root/lab27/before.sha256
```

```
            State : clean, degraded
    Active Devices : 1
/mnt/raid/database.bin: OK
/mnt/raid/ledger.txt: OK
```

6. Replace the failed disk and rebuild the mirror

Add the spare; mdadm rebuilds the mirror onto it automatically.

```
mapfile -t LO < /root/lab27/loops.txt
mdadm /dev/md10 --add "${LO[2]}"
cat /proc/mdstat
mdadm --wait /dev/md10 2>/dev/null; true
mdadm --detail /dev/md10 | grep -E 'State :|Active Devices|Working Devices'
```

The `State` returns to `clean` with 2 active devices — the array is redundant again.

7. Build a RAID 5 array and compare capacity

Three 128 MB disks in RAID 5 give about 256 MB usable; the two-disk mirror gives about 128 MB.

```
mapfile -t LO < /root/lab27/loops.txt
yes | mdadm --create /dev/md11 --level=5 --raid-devices=3 --run "${LO[3]}" "${LO[4]}"
"${LO[5]}"
mdadm --wait /dev/md11 2>/dev/null; true
for md in md10 md11; do
  echo "$md: $(mdadm --detail /dev/$md | grep -E 'Raid Level|Array Size' | tr -s ' ' | tr
'\n' ' ')"
done
lsblk -o NAME,SIZE,TYPE /dev/md10 /dev/md11
```

8. Show that RAID is not a backup, and save the configuration

Corrupt the ledger the way ransomware or a faulty application would — the mirror faithfully copies the damage to both disks. Then save the array definition for recovery documentation.

```
echo "CORRUPTED BY RANSOMWARE" > /mnt/raid/ledger.txt && sync
sha256sum -c /root/lab27/before.sha256
mdadm --detail /dev/md10 | grep 'State :'
mdadm --detail --scan | tee /root/lab27/mdadm.conf.sample
```

```
/mnt/raid/database.bin: OK
/mnt/raid/ledger.txt: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
             State : clean
```

The array is perfectly healthy and the ledger is still wrong: only a backup (Lab 26) can bring the old version back.

Test it

```
mdadm --detail /dev/md10 | grep -E 'State :|Active Devices'
grep database.bin /root/lab27/before.sha256 | sha256sum -c
mdadm --detail /dev/md11 | grep -E 'Raid Level'
```

Expected: `State : clean` with `Active Devices : 2` after the rebuild, `database.bin: OK`, and `Raid Level : raid5`.

Troubleshooting

Symptom	Fix
`mdadm: device /dev/md10 exists` or `... is busy`	The lab was run before; check `cat /proc/mdstat` and continue from the next step, or start a fresh playground.
`mdadm: RUN_ARRAY failed: Invalid argument`	The RAID kernel module is missing; run `modprobe raid1` (or `raid456` for RAID 5) and retry.

Symptom	Fix
`losetup: cannot find an unused loop device`	Run `modprobe loop`, then repeat step 2.

Challenge

Fail two members of the RAID 5 array one after the other and record what happens after the second failure. Then explain, using RAID 6 and RAID 10, how you would design storage for a database that must survive two simultaneous disk failures, and what each option costs in usable capacity.

Reflection

A manager says "we have RAID 1, so we don't need backups". Using step 8, explain which threats RAID handles and which it cannot, and how replication, snapshots and offsite backups fill the gap.

Exam tips

- RAID 0 = striping, no redundancy; RAID 1 = mirroring; RAID 5 = parity, survives one disk; RAID 6 = double parity, survives two; RAID 10 = mirrors plus striping.
- RAID provides availability and fault tolerance, not backup — it replicates corruption and deletion.
- High availability removes single points of failure at every layer: disks (RAID), servers (clustering/load balancing), power (UPS/generator), sites (hot/warm/cold).
- A degraded array is one failure away from data loss — monitor it and rebuild quickly.

References

- Linux RAID wiki — https://raid.wiki.kernel.org/
- mdadm manual — https://man7.org/linux/man-pages/man8/mdadm.8.html
- NIST SP 800-34 Rev. 1 Contingency Planning Guide — https://csrc.nist.gov/pubs/sp/800/34/r1/upd1/final
- Ubuntu Server documentation (software RAID) — https://documentation.ubuntu.com/server/
